方格取数 HDU1565

题目描述

给你一个 $n*n$ 的格子的棋盘，每个格子里面有一个非负数。 从中取出若干个数，使得任意的两个数所在的格子没有公共边，就是说所取的数所在的 2 个格子不能相邻，并且取出的数的和最大。

样例输入

```
3
75 15 21
75 15 28
34 70 5
```

样例输出

```
188
```

分析

模拟样例对应的取数方案

用二进制表示每个数取或不取 2^{n*n}

要用DP来做，先找到问题的阶段，一行一行取数，这是多阶段决策过程问题，过程的状态必须具备无后效性，如何理解呢，对于本一层，只从上一层的最优值选取就行了，而不用看再之前的

为了这种无后效性原则，每行的状态必须枚举到所有的情况，按照题意，合法状态（左右不冲突）只有这些

```
000 001 010 100 101
```

根据上述思路设计如下

```
for i=1 to n//多阶段决策，遍历问题阶段
    枚举第i行所有可行选取
        枚举第i-1行所有可行选取
        验证i和i-1是否冲突
            状态转移
```

状态转移时，如何避免同一行相邻共取1的冲突，可以采用位运算i & (i>>1)

```
    num=0;
    for (int i=0;i<1<<n;i++){
        if ((i&(i>>1))==0)
            b[++num]=i;
    }
```

状态转移时，如何避免与上一行同列共取1的冲突，利用位运算&

位运算操作

```
&符号，x&y，会将两个十进制数在二进制下进行与运算(都1为1，其余为0）然后返回其十进制下的值。例如
3(11)&2(10)=2(10)。
|符号，x|y，会将两个十进制数在二进制下进行或运算（都0为0，其余为1）然后返回其十进制下的值。例如
3(11)|2(10)=3(11)。
^符号，x^y，会将两个十进制数在二进制下进行异或运算（不同为1，其余为0）然后返回其十进制下的值。例如
3(11)^2(10)=1(01)。
~符号，~x，按位取反。例如~101=010。
<<符号，左移操作，x<<2，将x在二进制下的每一位向左移动两位，最右边用0填充，x<<2相当于让x乘以4。
>>符号，是右移操作，x>>1相当于给x/2，去掉x二进制下的最右一位
上述的组合操作，例如判断二进制下是否有相邻位共同选用了1的情况，(x & (x>>1))==1，相当于x转换成二进制后每
位和左边位是否共同选了1
```

状压DP注意点：

位运算要注意运算优先级，该加的括号一定要加

本题状态DP思路整理：

状态：

$dp[i][j]$ 表示前 i 行里，第 i 行选取方案为 j 时能够取得的最大数和

状态转移：

$dp[i][j] = max(dp[i][j], dp[i-1][k] + w[k][j])$

初值：

默认即可，第0行都是0

终值：

max$\{dp[n][j]\}$

代码：

```
#include<bits/stdc++.h>
using namespace std;
int n,num;
int a[21][21],dp[21][20000],b[20000];
int getv(int i,int x){
    int ans=0;
    int t=1;
    while(x){
        if (x & 1) ans+=a[i][t];
        x/=2;
        t++;
    }
    return ans;
}

int main(){
```

```
    while(~scanf("%d",&n)){
    memset(dp,0,sizeof(dp));
    num=0;
    for (int i=0;i<1<<n;i++){
        if ((i&(i>>1))==0)
            b[++num]=i;
    }
    for (int i=1;i<=n;i++)
        for (int j=1;j<=n;j++)
            scanf("%d",&a[i][j]);
    for (int i=1;i<=n;i++)
        for (int j=1;j<=num;j++){
            int k=b[j];
            int tmpv=getv(i,k);
            for (int j2=1;j2<=num;j2++){
                if ((k & b[j2])==0)
                    dp[i][j]=max(dp[i][j],dp[i-1][j2]+tmpv);
            }
        }
    int ans=-1;
    for (int i=1;i<=num;i++)
        ans=max(ans,dp[n][i]);
    printf("%d\n",ans);
    }
}
```

练手题：

$zhima111$ 导游

寻找飞行棋（zhima772）

入门组模拟比赛#6第四题

http://10.49.12.60/blogof/jcsy01/blog/915

题意描述

在矩阵中的选取情况转成二进制后，每一行每一列都只能有1个选取

解题思路

如果用 $dp[i][j]$ 表示第 i 个人选了 j 号棋子的方案数，发现转移的时候无法保证每个人选的棋子都不一样。

发现一共只有 4 行，可以考虑状压。我们用二进制来表示 4 个人选棋子的状态，显然一共有 16 种状态。用 $dp[i][j]$ 表示第 i 号棋子状态为 j 时候的方案数，我们可以横着递推。 对于第 i 号棋子，我们枚举 16 种状态，显然从 $i-1$ 号棋子转移过来的时候，假设当前状态为 $s1$ ，上一状态为 $s2$ 。如果能从 $s2$ 转移过来，首先 $s1$ 与 $s2$ 只能有一个位置不同，并且这个位置 pos 在 $s2$ 上必须为 0 ，在 $s1$ 上必须为 1 ，并且 $a[pos][i]$ 必须为 1 保证这个位置有棋子。那么 $dp[i][j]+=dp[i-1][k]$

最后我们的答案就是 $dp[n][15]$ 。时间复杂度为 O（ $n*16^2$ ）。

```
#include<bits/stdc++.h>
using namespace std;
int n;
long long ans,dp[5005][16];
char a[5][5005],vis[5005];
int main()
{
    cin>>n;
    for(int i=1;i<=4;i++){
        for(int j=1;j<=n;j++)cin>>a[i][j];
    }
    dp[0][0]=1;
    for(int i=1;i<=n;i++){
        for(int st0=0;st0<16;st0++){
            dp[i][st0]=dp[i-1][st0];
            for(int st1=0;st1<16;st1++){
                int sta=st0^st1;
                int t=0,k=0,pos=0;
                while(sta){
                    if(sta&1)t++,pos=k;
                    k++;
                    sta>>=1;
                }
                if(t!=1)continue;
                if(a[pos+1][i]=='0')continue;
                if( ( st0&(1<<pos) )  && ( st1&(1<<pos) )==0){
                   // cout<<st0<<' '<<' '<<st1<<' '<<dp[i-1][st1]<<endl;
                    dp[i][st0]+=dp[i-1][st1];
                }
            }
        }
    }
  //  cout<< (1<<1) <<endl;
    cout<<dp[n][15]<<endl;
    return 0;
}
```

POJ2817

题目

给定 $n(n \leq 10)$ 个字符串，排一个顺序，前后两个字符串根据题目描述的方法匹配出一个值，求最大的匹配值和。

两个字符串的匹配值，可以在其中一个字符串最前面加任意空格，对齐看相同字符数量

样例输入

```
5
abc
bcd
cde
aaa
bfcde
0
```

样例输出

```
8
```

分析

任意两个字符串匹配值可以预处理出来

状态需要描述哪些信息：字符串已经选了哪些；最后一串选了哪个

状态如何描述：

- 用二进制描述每个字符串目前是否已经选进来，$(1 << n) - 1$ 种情况
- 最后一串的编号
- $dp[k][i]$ k表示选用对应的二进制数的值，i表示最后一个选进来的字符串编号

问题阶段：

不像矩阵取数那么明显（一行一行来），因为谁是第一个谁是最后一个都未知

那就以已选用了几个字符串作为问题阶段

状态转移：

选用一个字符串的状态有这些：00001,1 00010,2 00100,3 01000,4 10000,5

00001,1-->00011,2 00101,3 01001,4 10001,5

……

总结上述

```
dp[k][i] --> dp[k|1<<j-1][j]  (k&i  && !(k & 1<<j-1))
```

伪代码：

```
for ii=1 to n //问题阶段，已选用几个字符串
    for k=0 to (1<<n)-1 //遍历选用字符串选用方案对应的二进制数
        for i=1 to n //上一个选用的字符串编号
            for j=1 to n  //下一个选用的字符串编号
                if(k & i && !(k & 1<<j-1))//枚举的选用方案要满足i有j没有


                    dp[k|1<<j-1][i]=max(dp[k|1<<j-1][i],dp[k][j]=w[i][j])//w[i][j]表示字符串i和j
```

```
的匹配值
```

注意点、易错点：

运算优先级、位运算用错、多组数据清空、如何调试

代码

```
#include<cstdio>
#include<iostream>
#include<cstring>
using namespace std;
int n,dp[1<<10][11],w[11][11];
char a[11][30];
void getw(int x,int y){
    int L1=strlen(a[x]+1);
    int L2=strlen(a[y]+1);
    w[x][y]=w[y][x]=0;
    for (int i=1;i<=L1;i++)
        for (int j=1;j<=L2;j++)
        {
            int cnt=0;
            int k=0;
            while (i+k<=L1&&j+k<=L2){
                if (a[x][i+k]==a[y][j+k]) cnt++;
                k++;
            }
            if (cnt>w[x][y])
                w[x][y]=w[y][x]=cnt;
        }
}
int main(){
    while (~scanf("%d",&n)){
        memset(dp,0,sizeof(dp));
        if (n==0) break;
        for (int i=1;i<=n;i++){
            scanf("%s",a[i]+1);
        }
        for (int i=1;i<n;i++)
            for (int j=i+1;j<=n;j++)
                getw(i,j);
        for (int i=1;i<=n;i++)
            for (int j=0;j<1<<n;j++)
                for (int k1=1;k1<=n;k1++)
                     if (j & 1<<k1-1)
                        for (int k2=1;k2<=n;k2++)
                            if (!(j & 1<<k2-1))
                                dp[j|1<<k2-1][k2]=max(dp[j|1<<k2-1][k2],dp[j][k1]+w[k1][k2]);
        int ans=-1;
        for (int i=1;i<=n;i++)
            ans=max(ans,dp[(1<<n)-1][i]);
        printf("%d\n",ans);

    }
```

```
}
```

POJ1170

题目

有最多5种物品，给出每个物品的单价。

给出几个这些物品的组合和这个组合的价格。买组合要比一件件的买便宜。

问给定的购买计划最少花多少钱。

样例输入

```
2
7 3 2
8 2 5
2
1 7 3 5
2 7 1 8 2 10
```

样例输出

```
14
```

分析

最多5种物品，每种最多5件，我们就可以把每种物品的选用压缩成二进制表示

每种组合选0个1个.....多个，就相当于做个完全背包

通过这道题目，我们可以看到，状态压缩不一定压缩成二进制，但能表示成二进制最好，因为可以使用位运算速度快。

代码

```
#include<cstdio>
#include<iostream>
#include<cstring>
using namespace std;
int a[2000],v[2000],b[20],dp[50000],c[2000];

bool cmp(int x,int y){
```

```
    while(x>0||y>0){
        if (x%6<y%6) return false;
        x/=6;
        y/=6;
    }
    return true;
}
int main(){
    b[0]=1;
    int tot=0,n,m;
    for (int i=1;i<=6;i++)
        b[i]=b[i-1]*6;//5位6进制权值
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        int id,num;
        scanf("%d%d%d",&id,&num,&v[i]);
        a[id]=i;//物品代号对应的读入编号
        c[i]=b[i-1];//c数组记录物品容量
        tot+=num*b[i-1];//n种物品总容量，相当于最终要取到的
    }
    scanf("%d",&m);
    for (int i=n+1;i<=n+m;i++){
        int T;
        scanf("%d",&T);
        int tmp=0;
        for (int j=1;j<=T;j++){
            int id,num;
            scanf("%d%d",&id,&num);
            tmp+=c[a[id]]*num;//计算每种套餐的容量
        }
        c[i]=tmp;//同上
        scanf("%d",&v[i]);

    }
    memset(dp,0x3f,sizeof(dp));
    dp[0]=0;
    for (int i=1;i<=n+m;i++){
        for (int j=0;j<=tot;j++){
            if (cmp(j,c[i]))  //第i种物品对应的容量c[i]要能被容量j完全涵盖
                dp[j]=min(dp[j],dp[j-c[i]]+v[i]);//完全背包最小值更新
        }
    }
    printf("%d\n",dp[tot]);
}
```

POJ2411

http://poj.org/problem?id=2411

题目

$n*m$（$1 \le n, m \le 11$）的方格矩阵，用 $1*2$ 的多米诺骨牌去填充，问有多少种填充方法。

分析

如何进行状态压缩

横铺用0表示，竖铺用1表示？

横铺用11表示，竖铺处在上方的是0，处在下方的是1？

可以有多种方式，只要你的状态描述能够涵盖所有情况，并且能够支持状态转移

然后分行进行多阶段决策。

比如6列的方格矩阵

1 1 0 0 1 1 //第一行横着放两个1*2，中间两个往下挂竖放

对于这样的第一行，第二行能怎样填？(画个图试试)

```
110011
001100

110011
111100

110011
001111

110011
111111
```

总结一般规律①：

上一行同一列如果为0，下一行就肯定是1

上一行同一列如果为1，下一行可能是0或1

- 上一行是1，下一行是0
- 上一行是1，下一行是1，并且要有偶数个连续1

解题思路一：

$dp[i][j]$ 表示前i行已放好，第i行对应的状态值是j的方案数

对于110011就是 $dp[1][110011]$ <====> $dp[1][51]$

状态转移： $dp[i-1][k]$ --> $dp[i][j]$ （k和j对应的二进制要满足规律①）

初值： $dp[0][(1<<m)-1]=1$

终值：$dp[n][(1<<m)-1]$

注意点：解会否爆int，要有这个意识

再思考，这样的方式高效吗？

对于列数m确定的情况下，任何两行的兼容关系都是一样的

第二行为110011，第三行只能是001100、111100、001111、111111

第三行为110011，第四行也只能是001100、111100、001111、111111

所以存在遍历上下行状态时存在很多无效遍历

因此我们想到，找出所有两行的兼容对应关系，存放在数组中，状态转移时直接遍历这个数组即可

如何找出这种兼容对应关系，我们可以使用搜索

```
dfs(int i,int x,int y)//i表示从左往右数当前构造第几位，x表示上一行前i位的状态，y表示下一行前i位状态
    递归边界，用g[count][0]=x;g[count][1]=y来存放兼容对应关系
    dfs(i+1，？，？) //根据规律①
```

解题思路二：

状态： $dp[i][j]$ 表示 前 i 行已经构造完，第 i 行状态为 j 的方案数

状态转移：

$k = g[count][0]$; $j = g[count][1]$

$dp[i][j]$ = $dp[i][j]$ + $dp[i-1][k]$ ($1 \le i \le n$)

初值：

$dp[0][(1 << m) - 1] = 1$

终值：

$dp[n][(1 << m) - 1]$

代码

```
#include<cstdio>
#include<iostream>
#include<cstring>
using namespace std;
int c,n,m;
int g[6000000][2];
long long dp[12][3000];
void dfs(int i,int x,int y){
    if (i>m) return;
    if (i==m){
        g[++c][0]=x;
        g[c][1]=y;
        return;
    }
    dfs(i+1,x<<1,y<<1|1);
    dfs(i+1,x<<1|1,y<<1);
    dfs(i+2,x<<2|3,y<<2|3);

}
```

```c
int main(){
    while (~scanf("%d%d",&n,&m)){
        if (n==0&&m==0) break;
        memset(dp,0,sizeof(dp));
        c=0;
        dfs(0,0,0);
        dp[0][(1<<m)-1]=1;
        for (int i=1;i<=n;i++)
            for (int j=1;j<=c;j++){
                dp[i][g[j][1]]+=dp[i-1][g[j][0]];
            }
        printf("%lld\n",dp[n][(1<<m)-1]);
    }
}
```